Unit I

The Nature of Software, Software Engineering, The Software Process

"Software Engineering taught us how to build software correctly. DevOps teaches us how to deliver software continuously, reliably, and at scale."

Why Software Engineering & DevOps?

- Software runs everything today
- Software changes continuously
- Failures are costly

Example: Online Exam System, Banking Apps, Food Delivery Apps

WHY SOFTWARE MATTERS

"When software succeeds, it can and does change things for the better. When software fails, bad things can and do happen." — Roger Pressman

- **Pervasive in Society**
  - Embedded in transportation (Ather Energy scooters, Tesla vehicles)
  - Medicine (diagnostic software, pacemakers)
  - Telecom (5G networks, carrier systems)
  - Military (defence systems, surveillance)
  - Industry (manufacturing ERP, IoT systems)
  - Entertainment (Netflix, gaming platforms)

What is Software?

- software is more than just code. It is a logical rather than a physical system element. It comprises:
  - Instructions (computer programs) that provide desired features, function, and performance when executed.
  - Data structures that enable the programs to adequately manipulate information.
  - Descriptive information (documentation) in both hard copy and electronic forms that describes the operation and use of the programs
- Example: Think of a Banking App. The *Instructions* allow you to transfer money; the *Data Structures* keep track of your balance and transaction history; the *Descriptive Information* is the "Help" section or user guide that explains how to set up two-factor authentication.

What is Engineering?

- Engineering is the application of a process, a collection of methods, and tools to build a high-quality product.
- It moves away from "trial and error" toward a disciplined, structured approach.
- Examples: Civil engineering builds bridges safely. Electrical engineering designs power systems.

Software Engineering

Core Definition

- *The application of a systematic, disciplined, and quantifiable approach to the analysis, design, implementation, and maintenance of software. It encompasses methods, tools, and processes to develop high-quality software that meets user requirements, is delivered on time, and within budget.* - **IEEE (1990)**

Or

Application of systematic, disciplined, quantifiable approach to development, operation, and maintenance of software "

- Systematic: Follows structured methodologies (Waterfall, Agile, Spiral)
- Disciplined: Applies proven techniques and standards
- Quantifiable: Measures quality, progress, and productivity
- Focus: Quality, reliability, maintainability, cost-effectiveness

ICSM-C 24B

THE NATURE OF SOFTWARE

- Today, software takes on a **dual role**. It is a product, and at the same time, the vehicle for delivering a product. As a product, it delivers the computing potential embodied by computer hardware or more broadly, by a network of computers that are accessible by local hardware.
- As the vehicle used to deliver the product, software acts as the basis for the control of the computer (operating systems), the communication of information (networks), and the creation and control of other programs (software tools and environments).
- Software delivers the most important product of our time—information. It transforms personal data so that the data can be more useful in a local context; it manages business information to enhance competitiveness; it provides a gateway to worldwide information networks and provides the means for acquiring information in all of its forms.
- The role of computer software has undergone significant changes over the last half- century. Dramatic improvements in hardware performance, profound changes in computing architectures, vast increases in memory and storage capacity, and a wide variety of exotic input and output options, have all precipitated more sophisticated and complex computer-based systems. Sophistication and complexity can produce dazzling results when a system succeeds, but they can also pose huge problems for those who must build complex systems.

Software vs. Hardware

Software	Hardware
S/W is logical unit	H/W is physical unit
No spare parts for s/w	Spare parts for h/w exist
Problem statement may not be complete and clear initially	Problem statement is clearly mentioned at the beginning of development
Requirements may change with time	Requirements are fixed before manufacturing
Multiple copies(less cost)	Multiple copies(more cost)
Idealized and actual graph	Bathtub graph

Software has characteristics that are considerably different than those of hardware:

1. Software is developed or engineered: it is not manufactured in the classical sense. Although some similarities exist between software development and hardware manufacturing, the two activities are fundamentally different. In both activities, high quality is achieved through good design, but the manufacturing phase for hardware can introduce quality problems that are nonexistent (or easily corrected) for software. Software projects cannot be managed as if they were manufacturing projects.

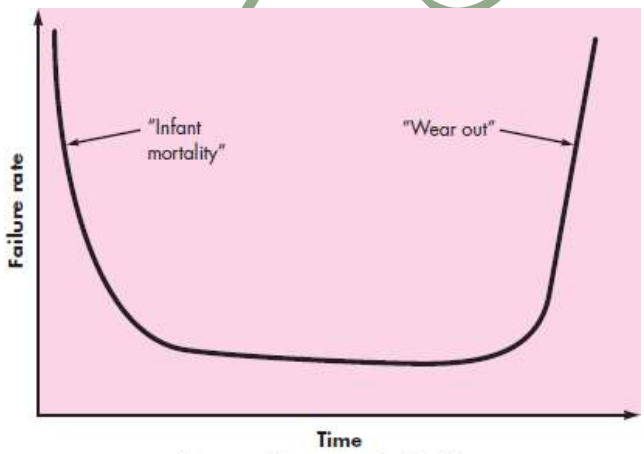


Fig 1.1: Failure Curve for hardware

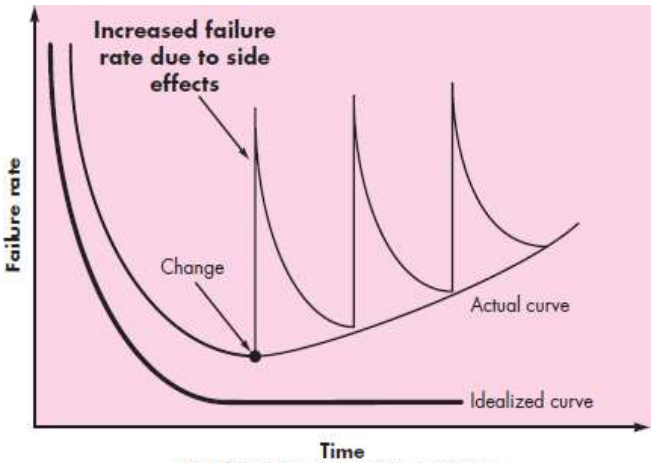


Fig 1.2: Failure curves for software

2. Software doesn't "wear out.": Figure 1.1 depicts failure rate as a function of time for hardware. The relationship, often called the "*bathtub curve*," indicates that hardware exhibits relatively high failure rates early in its life; defects are corrected and the failure rate drops to a steady-state level (hopefully, quite low) for some period of time. As time passes, however, the failure rate rises again as hardware components suffer from the cumulative effects of dust, vibration, abuse, temperature extremes, and many other environmental

SM-C24B

maladies.

Stated simply, the hardware begins to wear out. Software is not susceptible to the environmental maladies that cause hardware to wear out. In theory, therefore, the failure rate curve for software should take the form of the "idealized curve" shown in Figure 1.2. Undiscovered defects will cause high failure rates early in the life of a program. However, these are corrected and the curve flattens as shown.

The idealized curve is a gross oversimplification of actual failure models for software. However, the implication is clear—*software doesn't wear out. But it does deteriorate*.

3. **Although the industry is moving toward component-based construction, most software continues to be custom built:** As an engineering discipline evolves, a collection of standard design components is created. The reusable components have been created so that the engineer can concentrate on the truly innovative elements of a design, that is, the parts of the design that represent something new.

A software component should be designed and implemented so that it can be reused in many different programs. Modern reusable components encapsulate both data and the processing that is applied to the data, enabling the software engineer to create new applications from reusable parts.

SOFTWARE APPLICATION DOMAINS:

Today, seven broad categories of computer software present continuing challenges for software engineers:

- **System software** collection of programs written to service other programs. Some system software (e.g., compilers, editors, and file management utilities) processes complex, but determinate, information structures. Other systems applications (e.g., operating system components, drivers, networking software, telecommunications processors) process largely indeterminate data.
- **Application software**—stand-alone programs that solve a specific business need. Applications in this area process business or technical data in a way that facilitates business operations or management/technical decision making.
- **Engineering/scientific software**—has been characterized by "number crunching" algorithms. Applications range from astronomy to volcanology, from automotive stress analysis to space shuttle orbital dynamics, and from molecular biology to automated manufacturing.
- **Embedded software**—resides within a product or system and is used to implement and control features and functions for the end user and for the system itself.
- **Product-line software**—designed to provide a specific capability for use by many different customers. Product-line software can focus on a limited and esoteric marketplace or address mass consumer markets.
- **Web applications**—called "WebApps," this network-centric software category span a wide array of applications. In their simplest form, WebApps can be little more than a set of linked hypertext files that present information using text and limited graphics.
- **Artificial intelligence software**—makes use of nonnumerical algorithms to solve complex problems that are not amenable to computation or straightforward analysis. Applications within this area include robotics, expert systems, pattern recognition (image and voice), artificial neural networks, theorem proving, and game playing.

New Challenges

- **Open-world computing**—the rapid growth of wireless networking may soon lead to true pervasive, distributed computing. The challenge for software engineers will be to develop systems and application software that will allow mobile devices, personal computers, and enterprise systems to communicate across vast networks.
- **Net sourcing**—the World Wide Web is rapidly becoming a computing engine as well as a content provider. The challenge for software engineers is to architect simple and sophisticated applications that provide a benefit to targeted end-user markets worldwide.

UCSM-C24B

- **Open source**—a growing trend that results in distribution of source code for systems applications so that many people can contribute to its development.

Why does software need so many updates and changes?

- Software systems are built to represent or support the real world. The real world never remains constant. Rules change, technologies change, user expectations change, and environments change. Because of this, software must also change continuously.
- The answer is **software evolution**.

Meir Lehman (1974) formulated fundamental laws governing how software systems evolve over time. These are essential principles in software engineering.

- Lehman's laws are NOT rules
- They are observations about what actually happens to real software over time if, people keep using it

Software Evolution Laws

- **Law of Continuing Change**
  - A software system that represents real-world activities must be continuously changed. If it is not updated, it becomes progressively less useful.
  - **Example:**
    - A college ERP: it must update for new regulations and online exams; otherwise, it becomes unusable.
- **Law of Increasing Complexity**
  - As software evolves, its internal complexity increases unless work is done to reduce it.
  - Uncontrolled changes make software harder to understand and maintain.
  - **Example:**
    - A college ERP: Attendance + Exams + Results + Fees → More code → More complexity.
- **Law of Self-Regulation**
  - Software evolution processes are self-regulating and show predictable behavior.
  - Growth follows identifiable trends rather than random patterns.
  - **Example:**
    - A college ERP: Initially many updates, later only 2–3 updates per semester.
- **Law of Conservation of Organizational Stability**
  - The average rate of software development remains roughly constant over time. Adding more people does not proportionally increase productivity.
  - **Example:**
    - A college ERP: Adding more developers does not double progress due to meetings & coordination.
- **Law of Conservation of Familiarity**
  - Software evolution must preserve developer familiarity with the system. Excessive growth reduces understanding and increases errors.
  - **Example:**
    - A college ERP: If ERP grows too fast, developers get confused and make mistakes.
- **Law of Continuing Growth**
  - Software systems must continue to grow to satisfy users' changing needs. Without growth, user satisfaction decreases.
  - **Example:**
    - A college ERP: Parents portal → Mobile app → SMS alerts.
- **Law of Declining Quality**
  - The quality of software will decline unless it is actively maintained and adapted. Environmental and requirement changes reduce perceived quality.
  - **Example**
    - A college ERP: ERP feels slow on new servers without updates.
- **Law of Feedback System**
  - Software evolution is driven by feedback from users and developers. Feedback loops must be managed for effective evolution.
  - **Example:**

IICSM-C 24B

- A college ERP: Students complain → changes made → new version released.

Legacy Software

- Legacy software are older programs that are developed decades ago. The quality of legacy software is poor because it has inextensible design, convoluted code, poor and nonexistent documentation, test cases and results that are not achieved.
- As time passes legacy systems evolve due to following reasons:
  - The software must be adapted to meet the needs of new computing environment or technology.
  - The software must be enhanced to implement new business requirements.
  - The software must be extended to make it interoperable with more modern systems or database
  - The software must be re-architected to make it viable within a network environment.

What are legacy systems?

- Legacy systems include both the legacy software and the legacy hardware.
- Since the only change to the legacy system is replacement of the hardware with an industry standard server, the arduous process of software certification does not have to be repeated.

Who has legacy systems?

- Legacy systems are everywhere; banks, energy companies (including nuclear plants), manufacturing of all types (process control), the defense industry, transportation, hospitals, insurance, and more.

How to Modernize Legacy Systems

- likely use one of these three strategies:
- **Wrapping:** Surrounding the legacy code with a modern "API" layer. The old code stays, but it can now talk to mobile apps.
- **Re-factoring:** Restructuring the code without changing its external behavior.
- **Re-engineering:** The "Scrap and Rebuild" approach. This is the most expensive but often necessary when the hardware becomes obsolete.

Changing Nature of Software

Software Category	Description	How it evolved
System Software	Programs that service other programs (OS, compilers, drivers).	Shifting toward virtualization and containerization (Docker/K8s).
Application Software	Stand-alone programs for business needs (Word, POS systems).	Moving from desktop-based to **SaaS** (Software as a Service).
Engineering/Scientific	Complex numerical algorithms (CAD, simulations).	Now uses **Digital Twins** to simulate real-world physics in real-time.
Embedded Software	Resides inside a product (Microwave, Digital Watch).	Evolved into **IoT (Internet of Things)** everything is now connected.
Product-Line Software	Mass-market items (Office suites, graphics tools).	Focus is on **cloud-syncing** and collaborative real-time editing.
Web/Mobile Apps	Client-server applications.	Moving toward **Progressive Web Apps (PWAs)** and 5G integration.
AI Software	Uses non-numerical algorithms to solve complex problems.	Transitioning from "static models" to **Generative AI** and LLMs.

II CSM-C 24B

SOFTWARE ENGINEERING

Software engineering encompasses a process, methods for managing and engineering software, and tools. In order to build software that is ready to meet the challenges of the twenty-first century, few simple realities are:

- Software has become deeply embedded in virtually every aspect of our lives, and as a consequence, the number of people who have an interest in the features and functions provided by a specific application has grown dramatically.
- The information technology requirements demanded by individuals, businesses, and governments grow increasing complex with each passing year. The complexity of these new computer-based systems and products demands careful attention to the interactions of all system elements. It follows that design becomes a pivotal activity.
- Individuals, businesses, and governments increasingly rely on software for strategic and tactical decision making as well as day-to-day operations and control. If the software fails, people and major enterprises can experience anything from minor inconvenience to catastrophic failures.
- As the perceived value of a specific application grows, the likelihood is that its user base and longevity will also grow. As its user base and time-in-use increase, demands for adaptation and enhancement will also grow. It follows that software should be maintainable.

The IEEE has developed a more comprehensive definition when it states:

Software Engineering: (1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software. (2) The study of approaches as in (1).

Layered Technology:

Fig 1.3: Software Engineering layers

- Software engineering is a layered technology. Referring to Figure 1.3, any engineering approach (including software engineering) must rest on an organizational commitment to quality. Total quality management, Six Sigma, and similar philosophies foster a continuous process improvement culture, and it is this culture that ultimately leads to the development of increasingly more effective approaches to software engineering. The bedrock of software engineering is a ***quality focus***.
- The foundation for software engineering is the process layer. The software engineering process is the glue that holds the technology layers together and enables rational and timely development of computer software. Process defines a framework that must be established for effective delivery of software engineering technology.
- The *software* ***process*** forms the basis for management control of software projects and establishes the context in which technical methods are applied, work products (models, documents, data, reports, forms, etc.) are produced, milestones are established, quality is ensured, and change is properly managed.
- Software engineering ***methods*** provide the technical how-to's for building software. Methods encompass a broad array of tasks that include communication, requirements analysis, design modeling, program construction, testing, and support. Software engineering methods rely on a set of basic principles that govern each area of the technology and include modeling activities and other descriptive techniques.
- Software engineering ***tools*** provide automated or semi-automated support for the process and the methods. When tools are integrated so that information created by one tool can be used by another, a system for the support of software development, called computer-aided software engineering, is established.

IT-C-1-C 24B

The Software Process What is a Software Process?

- A software process defines **who does what, when, and how** to develop software.
- Software process means to develop quality products by using technical & management rules.

Every software project, irrespective of size or domain, follows five **generic framework activities**.

- **Communication**
  - Communication involves interacting with customers and stakeholders to understand their needs, expectations, and requirements.
  - **Scenario:** Discussing features and rules with faculty before developing an online exam system.
- **Planning**
  - Planning defines the work to be done, resources required, risks involved, and a schedule to complete the project.
  - **Scenario:** Preparing a project plan for a mobile app with timeline and team roles.
- **Modeling**
- Modeling helps in understanding requirements and system design by creating diagrams or representations before coding.
- **Scenario:** Drawing a login and payment flow diagram before developing an e-commerce app.
- **Construction**
  - Construction includes writing the code and testing it to ensure that errors are identified and fixed.
  - **Scenario:** Developing modules and performing unit testing in a banking application.
- **Deployment**
  - Deployment involves delivering the software to users and collecting feedback for improvements.
  - **Scenario:** Releasing a new version of an app on Play Store and fixing user-reported issues.
- These framework activities are not performed only once
- They are repeated in iterations
- Each iteration delivers a working version of software

Framework activities alone are NOT sufficient.

- Umbrella Activities are used to: Support, control, and improve all framework activities throughout the entire software project.

Umbrella Activities (Run Across All Phases)

- **Software Project Tracking & Control**
  - Tracks project progress against the plan and takes corrective action if deviations occur.
  - **Scenario:** Checking whether a mobile app project is on schedule and adjusting resources if delayed.
- **Risk Management**
  - Identifies and analyzes risks that may affect the project and plans actions to reduce their impact.
  - **Scenario:** Identifying server failure risk before deploying an online exam system.
- **Software Quality Assurance (SQA)**
  - Ensures that software development follows defined standards and procedures to maintain quality.
  - **Scenario:** Ensuring coding standards and testing practices are followed in a banking app.
- **Technical Reviews**
  - Reviews software work products to detect errors early before they move to the next stage.
  - **Scenario:** Reviewing design diagrams before starting coding
- **Measurement**
  - Collects data related to process, project, and product to improve planning and quality.
  - **Scenario:** Measuring number of defects found per testing cycle.
- **Software Configuration Management (SCM)**
  - Manages and controls changes made to software to avoid confusion and errors.
  - **Scenario:** Using version control to track changes in source code.
- **Reusability Management**
  - Identifies and manages reusable components to reduce development time and cost.

IICSM-C 24B

- **Scenario:** Reusing an authentication module across multiple applications.
- **Work Product Preparation & Production**
  - Involves preparing documents, reports, models, and manuals required during software development.
  - **Scenario:** Preparing user manuals and test reports for a released system.

Software Engineering Practice

describes how engineers solve problems while performing software process activities.

- Process = what to do
- Practice = how to do it correctly

Software Engineering Practices

- Software engineering practice consists of four fundamental activities:
  - Understand the problem
  - Plan a solution
  - Carry out the plan
  - Examine the result for accuracy

Example: **Online Examination System**

1. **Understand the Problem** (Communication and Analysis)
   It's worth spending a little time to understand, answering a few simple questions:
   - Before starting development, the engineer clearly understands:
   - Who will use the system (students, faculty, admin)
   - What the system should do (conduct exams, evaluate answers)
   - What rules apply (time limits, grading logic)
2. **Plan a Solution** (Modeling and Software Design)
   - Now you understand the problem and you can't wait to begin coding
   - After understanding the problem, the engineer plans:
     - System structure (modules for login, exam, evaluation)
     - Data design (questions, answers, marks)
     - Flow of operations
3. **Carry Out the Plan** (Code Generation)
   - Now you understand the problem (or so you think) and you can't wait to begin coding. Before you do, slow down just a bit and do a little design:
   - Now the engineer:
     - Writes code according to the design
     - Implements planned modules
     - Follows coding standards
4. **Examine the Result for Accuracy** (Testing and Quality Assurance)
   - You can't be sure that your solution is perfect, but you can be sure that you've designed a sufficient number of tests to uncover as many errors as possible.
   - Finally, the engineer checks:
     - Whether exams start and end correctly
     - Whether marks are calculated properly
     - Whether requirements are satisfied

Software Engineering Principles?

- Software engineering principles are **fundamental guidelines** that help engineers **make correct decisions** while developing software.
- All principles ultimately support **quality, maintainability, and value**.
- David Hooker has proposed seven principles that focus on software engineering practice as a whole. They are as following.

1. The Reason It All Exists
   - A software system exists **only to provide value to its users**. If a feature or requirement does not add value, it should not be included.
2. Keep It Simple, Stupid!

IICSM-C 24B

- Software design should be **as simple as possible**, but no simpler. Simple designs are easier to understand, maintain, and less error-prone.

3. Maintain the Vision
   - A software project must have a **clear architectural vision**. Compromising this vision weakens the system and leads to failure.
4. What You Produce, Others Will Consume
   - Software should be designed and coded keeping **future developers in mind**. Readable, well-structured code makes maintenance and debugging easier.
5. Be Open to the Future
   - Software systems should be designed to **accommodate future changes**. Never design software in a way that restricts future enhancement.
6. Plan Ahead for Reuse
   - Designing components for reuse **reduces cost** and **increases value**. Reusable components save effort in future projects.
7. Think!
   - Careful thinking before action produces better results. Rushing into coding without thinking leads to mistakes.

Software Myths

- are **misleading beliefs** about software development that cause **poor decisions, unrealistic expectations, and failures**.
- **Management Myths**
  - **Myth 1**
    - "We already have standards and procedures; that's enough."
    - **Reality:** Standards are useful only if they are used, updated, and practical.
  - **Myth 2**
    - "If the project is late, we can add more programmers and catch up."
    - **Reality:** Adding people to a late project usually makes it later due to communication overhead.
  - **Myth 3**
    - "If we outsource the project, we can relax."
    - **Reality:** Without internal project understanding, outsourcing also fails.
- **Customer Myths**
  - **Myth 1**
    - "A general statement of objectives is enough to start coding."
    - **Reality:** Clear and continuous requirement communication is essential.
  - **Myth 2**
    - "Changes are easy because software is flexible."
    - **Reality:** Changes become more expensive as development progresses.

Practitioner (Developer) Myths

- **Myth 1**
  - "Once the program works, the job is done."
  - ***Reality:*** Most effort occurs after delivery during maintenance.
- **Myth 2**
  - "We can assess quality only after the program runs."
  - ***Reality:*** Reviews are more effective than testing for many defects.
- **Myth 3**
  - "The only important deliverable is working code."
  - ***Reality:*** Documents, designs, and test artifacts are equally important.
- **Myth 4**
  - **"**Software engineering creates unnecessary documentation and slows us down."
  - ***Reality:*** Good engineering reduces rework and actually speeds delivery.

Software project failures often occur not because of technology,but because basic principles are ignored and myths are believed

HCSM-C 24B

GENERIC PROCESS MODEL

What is a Process Model?

- A process model defines the order and flow in which software development activities are performed.

It explains:

- When an activity starts
- When it ends
- Whether activities are repeated or done together

A process model is a way of organizing software work

- **Why Process Models are Needed**
  - Software projects differ in: Size, Complexity, Requirements, Risk
  - Because of this:
    - One single development approach is not suitable for all projects
    - Different projects require different **process models**
    -

About SDLC Process

- SDLC represents the process of developing quality software as per customer requirements.

➢**Need of SDLC Process:**

1. Improved relation with client & development team.
2. It offers basic project planning, scheduling & cost estimation.
3. Provide standard set of activities & SE rules.
4. Increase & enhance development speed.
5. Maintain project tracking & control.
6. Decrease any type of project risk.
7. Decide entry & exit criteria of each phase.

SDLC Phase 1: Requirement Analysis & Planning

➢Phase conduct by Product manager, Business Analyst or senior members in team.

➢To understand the Aim, Purpose & Exact requirements of the customer.

➢Conduct market research, customer interviews & surveys.

➢Software Requirement Specification (SRS) document is created.

➢Identify risk assessment in project.

➢Planning of Project schedule, Cost estimation & other recourses.

Example: Online Shopping Application

Customer requirements like:

- Categories like Ladies wear, Gents wear module, Grocery module, Shopping cart module, My order module, Discount module, Notification module.
- Good & Attractive user interface.

SDLC Phase 3: Implementation

- Phase conduct by Developer or Programmer.
- Must follow predefine coding guidelines or discuss with management team.
- Used suitable Programming Languages like C, C++, JAVA, ANDROID, PYTHON, PHP etc.
- Used suitable Databases like SQL, ORACLE etc.
- Programming tools like Compilers, Interpreters, Debuggers etc.

Example:

Online Shopping Application

```
void customerDetails()
{
    cout << "Enter your name: ";
    string customer_name;
    getline(cin,customer_name );
    cout << "WELCOME ";
    for(int i = 0; i<customer_name.length(); i++ )
    {
        cout << char(toupper(customer_name[i] ));
    }
    cout << "\n";
}
```

SDLC Phase 4: Testing

- Phase conduct by QA Team or Tester.
- It perform Integration Testing, Black Box Testing, White Box Testing, System Testing, Regression Testing Performance Testing & Acceptance Testing etc.
- The testing team starts testing the functionality of the entire system.
- If find some bugs/defects which report to developers then Development team fixes the bug and send back to QA.
- This is done to verify that the entire application works according to the customer requirement.

Example:

Online Shopping Application

SDLC Phase 5: Deployment & Maintenance

- Phase conduct by Project Manager, Operation or Production support team.
- Once the software is certified and no bugs or errors are stated, then final software is release in market or deployed in customer environment.
- If any issue comes up and needs to be fixed or any updating is to be done is taken care by the Support Managers in Maintenance phase.

Example:

Online Shopping Application

All software projects follow the same framework activities:

- Communication, Planning, Modeling, Construction, Deployment
- Generic process models describe how these activities are arranged and repeated.

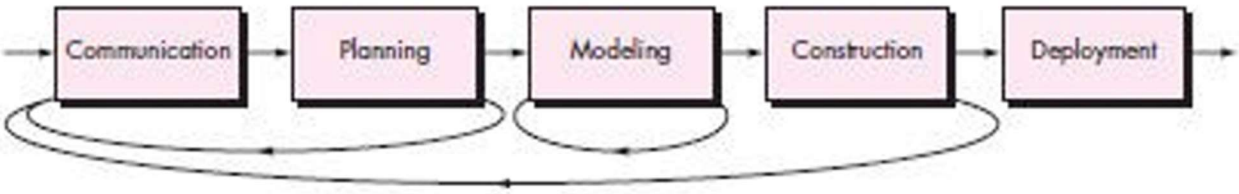
1. Linear Process Flow

- "Finish one activity completely, then move to the next."
- "First talk to customer, then plan, then design, then code, then deliver."
- No going back easily.

(a) Linear process flow

2. Iterative Process Flow

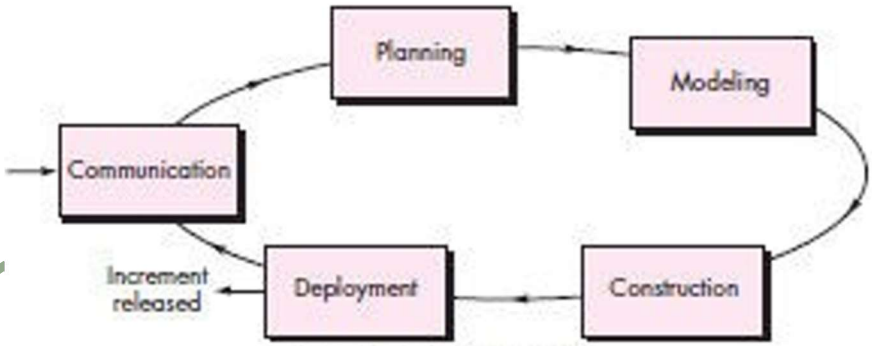
- "Do some work, check it, then improve it again."
- "Write part of the software, test it, take feedback, fix it, repeat."
- Same steps are repeated.



(b) Iterative process flow

3. Evolutionary Process Flow

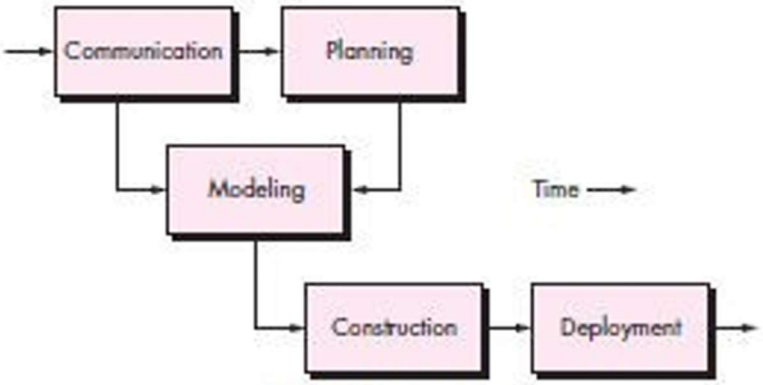
- "Don't build everything at once. Build small working parts and improve them."
- "First release basic version, then add features slowly."
- Software grows over time.



(c) Evolutionary process flow

4. Parallel Process Flow

- "Some activities are done at the same time."
- "While coding is going on, testing and planning for next part also continue."
- Saves time but needs coordination.



(d) Parallel process flow

Prescriptive Process Models

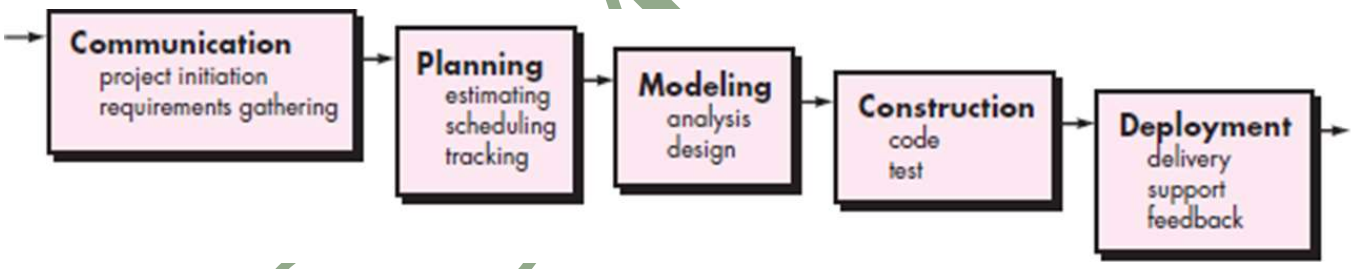
- Prescriptive process models are **traditional software process models** that prescribe:
  - A **specific set of framework activities**
  - A **defined order** in which these activities must be performed
  - Clearly defined **work products, milestones, and reviews**
- These models provide a **structured and disciplined approach** to software development.

Key Idea

  - All prescriptive models:
  - Use the **generic framework activities** (communication, planning, modeling, construction, deployment)
  - But apply **different emphasis and ordering** of these activities
  - The Waterfall Model
  - V-Model
- Incremental Process Models
- Evolutionary Process Models
  - **Prototyping**
  - **Spiral Model**
- Concurrent Models

WATERFALL MODEL

- The **Waterfall Model** (also called the *classic life cycle*) is a **linear and sequential process model**. It is suitable when **requirements are well understood and stable**.
- Development proceeds in a **step-by-step manner**, flowing downward like a waterfall.
- Systematic and disciplined approach
- Clear documentation at each stage
- Easy to manage due to well-defined phases

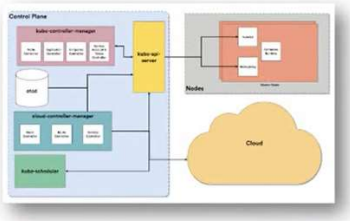


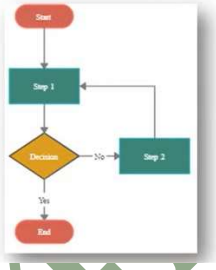
Communication

- The aim of this phase is to understand the exact requirements of the customer and to document them properly.
- They discuss functions, performance and interfacing requirement of the software.
- In this phase, Software Requirement Specification (SRS) document is created .
- It contained a detailed description of what the system will do in the common language.
- SRS document serves as a contract between the development team and customers.

Planning

- All gathered requirements are converted into suitable design.
- It defines the overall software architecture together with high level and detailed design.
- High level design include Algorithm, Flow charts, Decision tree, Database design etc.
- Low level design include Rough paper design, User interface components etc.
- Also finalized Programming languages, Database or other Hardware & Software requirements.
- All this work is documented as a Software Design Document (SDD).



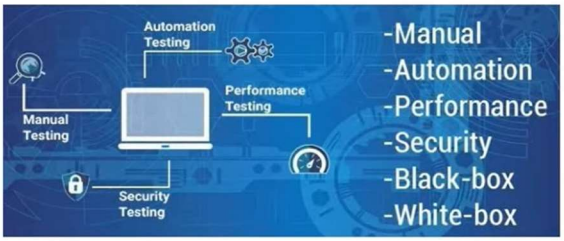


Modelling

- In this phase software design is translated into source code using any suitable programming language.
- The developer first developed in small programs called units after that integrated it.
- Unit testing is done in this phase to check whether each module is working properly or not.

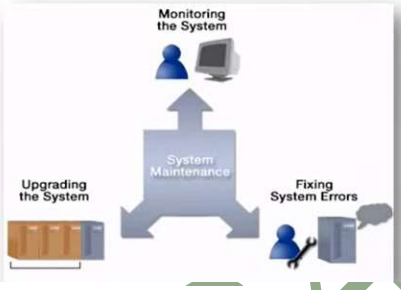
Construction

- In this phase, Tester perform all the testing activities to make sure that the system meets the client requirements or not
- After combining all the unit modules, Integration testing is done for any faults and failures.
- In case of any anomaly, report it.
- It generates Test cases & Test reports.



Deployment

- The product is deployed in the customer environment or released into the market.
- In maintenance phase, There are some issues which come up in the client environment.
- To fix those issues, patches are released.
- Also to enhance the product some better versions are released.
- Maintenance is done to deliver these changes in the customer environment.



Advantages of Waterfall Model

1. Simple and easy to understand and use
2. Before the next phase of development, each phase must be completed
3. Works well for smaller projects.
4. Requirements are very well understood & fixed.
5. Any changes in software is made during the process of the development
6. Process and results are well documented.

Disadvantages of Waterfall Model

1. High amounts of risk and uncertainty.
2. Error can be fixed only during the phase.
3. Not suitable for the projects where requirements are changing.
4. It becomes very difficult to move back to the phase.
5. Not a good model for complex and object-oriented projects.
6. Clients valuable feedback cannot be included with ongoing development phase

THE V MODEL

- The **V-Model** is a variation of the Waterfall Model.
- It emphasizes the **relationship between development phases and testing phases**.
- Left side of the "V" → Development activities
- Right side of the "V" → Corresponding testing activities.

Key Idea of V-Model

- For every development activity, there is a corresponding testing activity.

This ensures:

- Early error detection
- Strong quality control
- Reduced risk

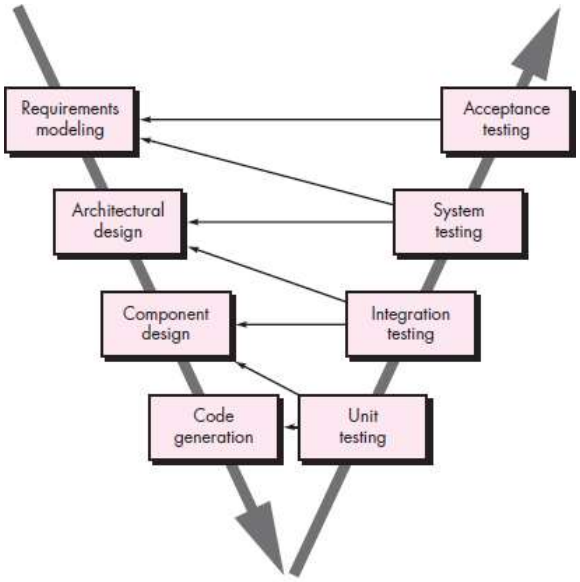


Fig 2.4 V - Model

When to Use the V-Model

- The V-Model is suitable when:
- Requirements are **clear, fixed, and well documented**
- The system is **safety-critical**
- Failure can cause **serious damage or loss**
- Strong **verification and validation** are required
- Regulatory or industry standards must be followed

Typical Domains Where V-Model is Used

- Airline reservation systems
- Medical devices software
- Automotive embedded systems
- Defense and aerospace systems

Advantages of V-Model

- Simple and easy to understand
- Testing is planned early
- Defects are detected early
- High quality and reliable software
- Suitable for safety-critical systems

Disadvantages of V-Model

- Rigid and inflexible
- Difficult to handle changing requirements
- No early working prototype
- High cost for small projects

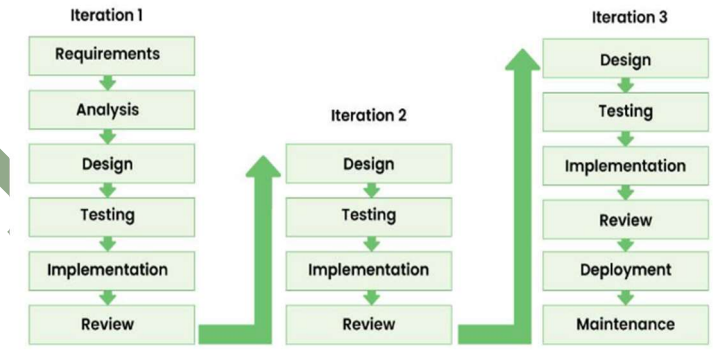
ITERATIVE MODEL

What is the Iterative Model?

- The **Iterative Model** develops software through **repeated cycles (iterations)**.
  The system is built, reviewed, improved, and rebuilt multiple times until it meets requirements.
- Each iteration refines the **same software**, not new parts

When to Use the Iterative Model

- Requirements are **not fully understood initially**
- Customer feedback is important
- The system is expected to evolve
- Risk can be reduced through repeated refinement



INCREEMENT MODEL

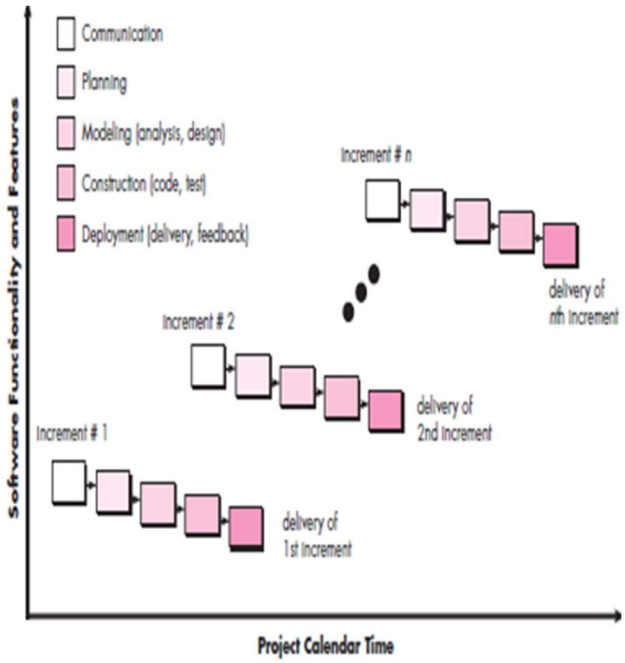
- The **Incremental Model** develops software by **breaking the system into smaller functional parts called increments.**
- Each increment is **fully designed, coded, tested, and delivered** before the next increment is started.
- The complete system is obtained by **adding increments step by step.**

Why Incremental Model

- Delivers **usable software early**
- Reduces risk by building **important features first**
- Allows feedback after each increment

When to Use the Incremental Model

- The Incremental Model is suitable when:
- Core requirements are **clearly known**
- Project is **large and can be divided into modules**
- Early delivery of basic functionality is required
- Risk needs to be managed gradually



CSMC 24B

Advantages of Incremental Model

- Early delivery of working software
- Reduced project risk
- Easy testing and debugging
- Customer feedback can be incorporated
- Suitable for large systems
- Better progress visibility

Drawbacks of Incremental Model

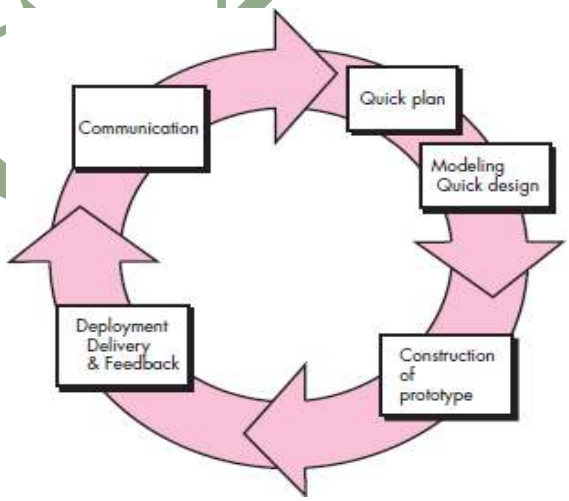
- Requires good initial architecture
- Integration between increments can be complex
- Total cost may increase
- Requires careful planning
- Not suitable if system cannot be modularized

EVOLUTIONARY PROCESS MODEL

- Evolutionary models are used when **requirements are not fully known at the beginning** and the software needs to **evolve gradually** based on feedback and risk analysis.
    - *Prototyping Model*
    - *Spiral Model*

PROTOTYPING MODEL

- The **Prototyping Model** builds a **quick working model (prototype)** of the software to understand requirements clearly.
- The prototype is shown to users, feedback is collected, and the system is **refined repeatedly** until final requirements are clear.
- Helps users **visualize the system early**
- Reduces requirement misunderstanding
- Improves communication between user and developer



When to Use the Prototyping Model

- Requirements are **not well defined**
- User interaction is important
- UI-intensive applications are developed
- Customer needs frequent clarification

Advantages of Prototyping Model

- Helps clarify requirements early
- Improves user involvement
- Reduces requirement-related risks
- Faster development of initial system
- Better user satisfaction

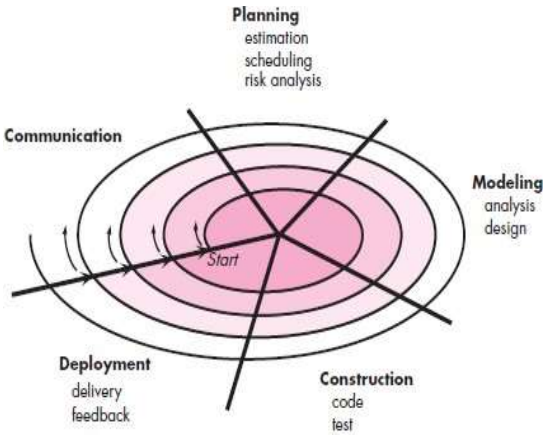
Drawbacks of Prototyping Model

- Prototype may be mistaken as final product
- Poor system design if prototype is reused
- Documentation is often ignored
- Frequent changes increase complexity
- Not suitable for large systems

SPIRAL MODEL

The **Spiral Model** is a **risk-driven evolutionary model** that combines:

- Iterative development
- Prototyping
- Risk analysis



Development proceeds in a **spiral of loops**, and each loop represents a phase

Why Spiral Model

- Identifies and resolves risks early
- Allows continuous refinement
- Suitable for high-risk projects
- preferred when **risk management is critical**.

IICSM-C 24B

When to Use the Spiral Model

- Project is **large and complex**
- High risk is involved
- Requirements change frequently
- Cost of failure is high

Advantages of Spiral Model

- Strong focus on risk analysis
- Flexible and iterative
- Early detection of issues
- Suitable for complex projects
- Continuous user involvement

Drawbacks of Spiral Model

- Complex to manage
- High development cost
- Requires risk analysis expertise
- Not suitable for small projects
- Difficult to schedule

Concurrent Models

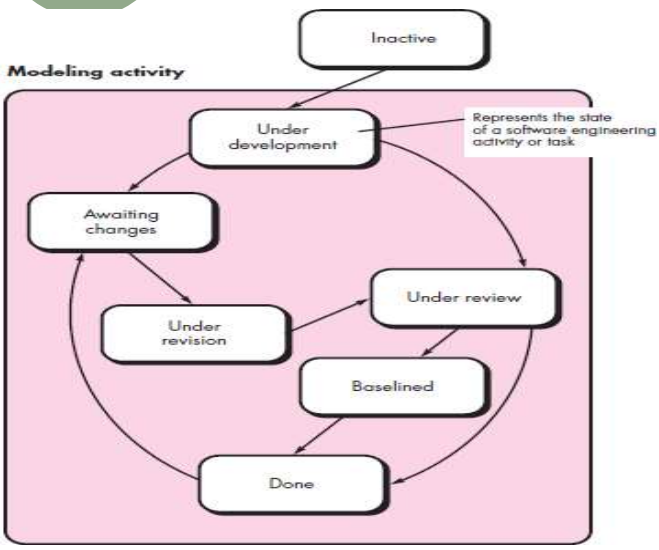
- The **Concurrent Process Model** is a software development model in which **multiple framework activities**—such as communication, planning, modeling, construction, and testing—**proceed simultaneously** rather than sequentially.

In this model:

- Each activity is treated as a **state-based process**
- Activities move through different states like *under development, under review, under revision,* etc.
- Progress in one activity can **trigger changes** in another activity.

When to Use the Concurrent Process Model

- The Concurrent Process Model is suitable when:
- The project is **large and complex**
- Multiple teams work on different parts
- Requirements change during development
- Activities need to overlap to reduce time
- Strong communication and coordination are possible



Example: **e-commerce project**

- **Communication** with stakeholders is ongoing
- **Planning** is updated continuously for new features
- **Modeling** (UI, database) is revised based on feedback
- **Construction** (coding) starts for stable modules
- **Testing** runs in parallel with coding

At the same time:

- UI design may be *under revision*
- Backend coding may be *under development*
- Testing may be *under review*
- All activities progress **concurrently**, not one after another.

THE UNIFIED PROCESS

- The **Unified Process** is a **modern software development model** that builds software **step by step (incrementally)** and **improves it repeatedly (iteratively)**, while **reducing risk early and managing complexity**.
- It organizes development into **four phases**, while continuously performing activities like **requirements, design, coding, and testing**.
- UP is **use-case driven**, **architecture-centric**, and **risk-focused**.

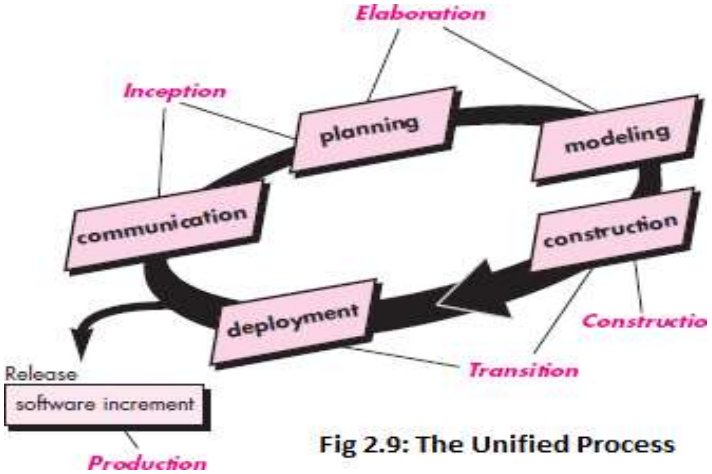


Fig 2.9: The Unified Process

1. Inception — *"Should we build it?"*

- Define scope and feasibility

IICSM-C24B

- Identify main use cases
- Rough cost and schedule

2. Elaboration — *"How should we build it safely?"*

- Architecture is designed and validated
- Major risks are reduced
- Detailed requirements refined

3. Construction — *"Build the system"*

- Features developed incrementally
- Coding and testing dominate
- Working software produced

4. Transition — *"Deliver to users"*

- System deployed
- User training
- Bug fixes and stabilization

When to Use the Unified Process

- Project is large and complex
- Object-oriented development is used
- Architecture is critical
- Risk must be managed early
- Long-term maintainability is required

Advantages of the Unified Process

- Iterative & incremental delivery
- Early risk reduction
- Strong architectural focus
- Handles requirement changes
- Suitable for enterprise systems

Drawbacks of the Unified Process

- Complex to understand/manage
- Requires experienced teams
- Higher management overhead
- Not ideal for small projects

Personal and Team Process Models

- Software engineering is not only about **process models** (Waterfall, Spiral, Unified Process), but also about **how individuals and teams actually work** while following those models.
- This idea leads to **Personal and Team Process Models**.

PSP (Personal Software Process)

- **PSP** is a **personal process model** that helps an **individual software engineer** plan, measure, and improve their own work.

PSP focuses on how a single developer writes better software.

What PSP Emphasizes

- Planning personal work
- Estimating time and size accurately
- Tracking defects
- Improving personal software quality

PSP assumes:

"If individuals improve their discipline, software quality improves."

Team Process Model – TSP (Team Software Process)

- **TSP** is a **team-level process model** that helps a **group of software engineers** plan, manage, and build software together effectively.
- **TSP focuses on how a software team works together.**

What TSP Emphasizes

- Team planning and scheduling
- Role assignment (leader, tester, designer)
- Team quality management
- Tracking progress and risks

TSP assumes:

"Good individuals must be organized into a disciplined team."

IICSM-C 24B

Quick Review of Models

Model Category	Model	Core Idea (1 line)	Strength	Typical Use	Main Limitation
Prescriptive	**Waterfall**	Development follows a strict sequence of phases	Simple, disciplined	Small, stable projects	Very rigid, no flexibility
	V-Model	Verification & validation planned alongside development	Strong testing focus	Safety-critical systems	Still rigid to change
Evolutionary	**Incremental**	Software delivered in small functional parts	Early working software	Large modular systems	Needs strong architecture
	Iterative	Same system refined through repeated cycles	Handles unclear requirements	Evolving requirements	Hard to estimate time/cost
	Spiral	Development driven by risk analysis	Excellent risk control	Large, high-risk systems	Complex, costly
Concurrent	**Concurrent Model**	Activities progress in parallel through states	Fast & flexible	Large team projects	Hard to manage
Unified	**Unified Process**	Iterative framework with strong architecture focus	Balanced discipline & flexibility	Large OO systems	Heavy for small projects

What is agile?

- Agile is a process that allows a team to more efficiently manage a project by breaking it down into several stages, each of which allows for consistent collaboration with stakeholders to promote steady improvements at every stage.
- In software development, agile practices involve discovering requirements and developing solutions through the collaborative effort of self-organizing and cross-functional teams and their customer/end user.

About Agile Model

About Agile:

- Mostly used model in today digital era.
- Agile means "The ability to respond to changes from requirements, technology & people"
- It is an incremental and iterative process of software development.

Working with Example:

- Divides requirements into multiple iterations & provide specific functionality for the release.
- Deliver multiple software requirements.
- Each iteration are lasts from two to three weeks.
- Direct collaboration with customers.
- Rapid project development.

Some of the reallife examples of agile model:

Restaurant orders:

- Preparation of some of the food before opening the shop (sprint planning)
- continuous delivery of orders
- number of successful orders

ICSM-C24B

cricket team:

- Run rate
- team (scrum team self-sufficient)
- over (sprint length)
- captain/ coach (scrum master)

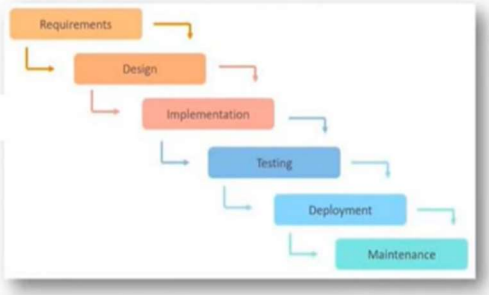
Traditional VS Agile Model Working with Example

For Example:

Instagram Social Application:

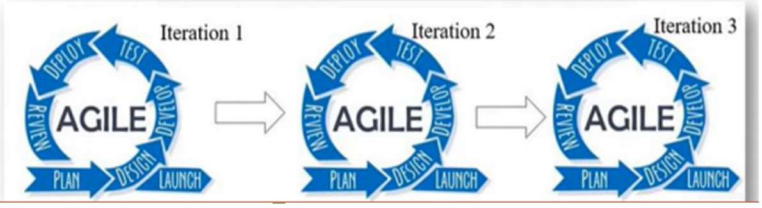
Requirements are:

1. Follow – Unfollow Option
2. Edit profile
3. Search
4. Messaging
5. Post photos
6. Upload story
7. To make reels
8. Go Live



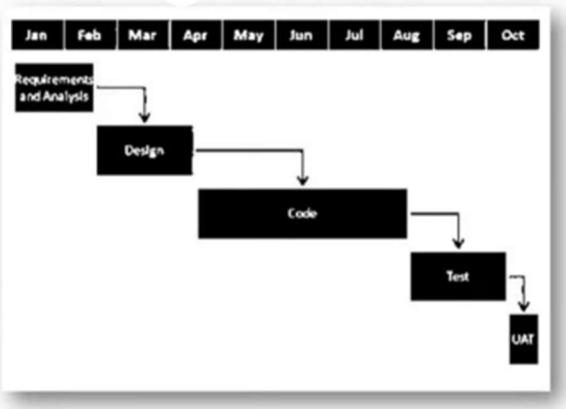
Waterfall Model

Agile Model

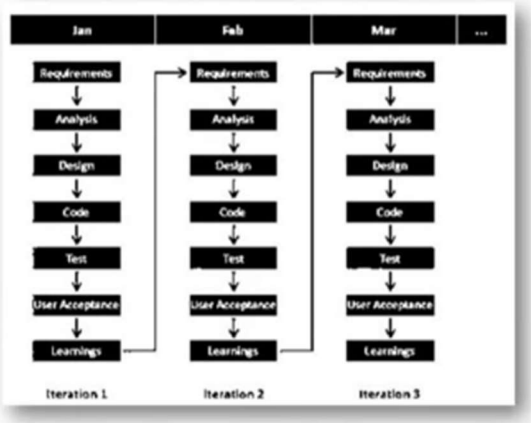


Traditional VS Agile Model Working with Example

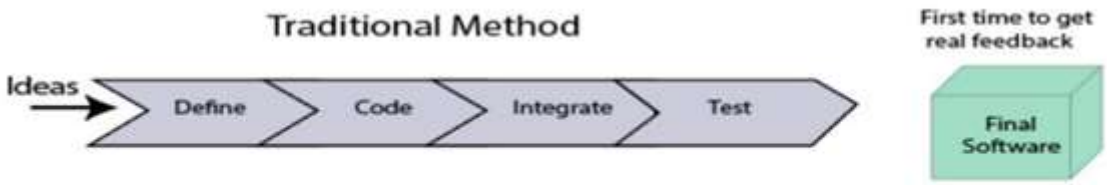
For Example:

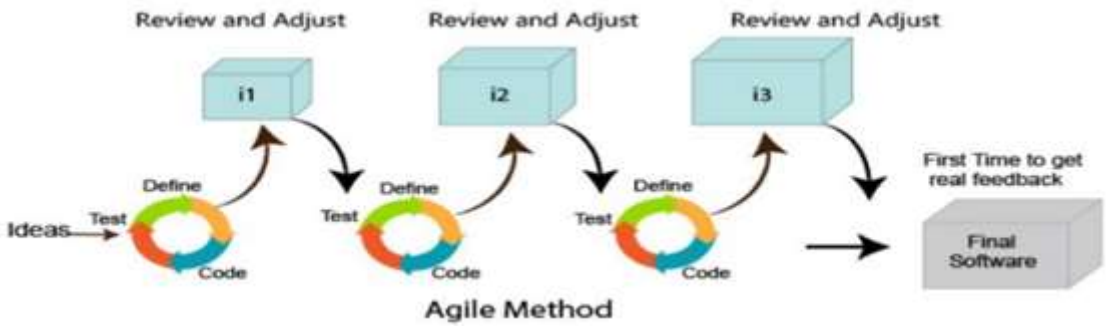


Waterfall Model
Time Span



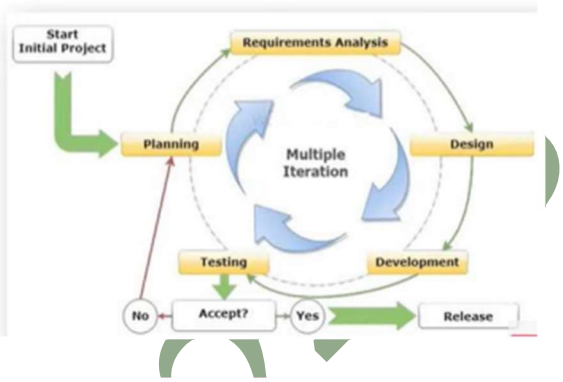
Agile Model
Time Span





When to use the Agile Model?

1. When project size is large.
2. When frequent changes are required.
3. When a highly qualified and experienced team is available.
4. When a customer is ready to have a meeting with a software team all the time.
5. Projects with flexible timelines and budget



Agile Methodology

Agile software development refers to a group of software development methodologies based on iterative development and incremental.

Agile development refers to any development process that is aligned with the concepts of the Agile Manifesto. The Manifesto was developed by a group of fourteen leading figures in the software industry, and reflects their experience of what approaches do and do not work for software development.

Advantages of agile model

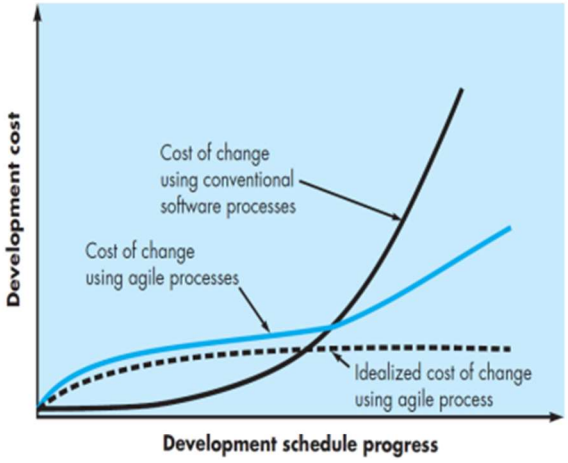
- Continuous delivery
- Continuous feedback
- Requirements changes in the middle
- Client satisfaction is very high
- Less development time
- Less development cost

Disadvantages of the Agile Model

- Not suitable for handling complex dependencies.
- More risk of sustainability, maintainability and extensibility.
- An overall plan, an agile leader and agile PM practice is a must without which it will not work.
- Strict delivery management dictates the scope, functionality to be delivered, and adjustments to meet the deadlines.
- Depends heavily on customer interaction, so if customer is not clear, team can be driven in the wrong direction.
- There is a very high individual dependency, since there is minimum documentation generated.
- Transfer of technology to new team members may be quite challenging due to lack of documentation.

AGILITY AND THE COST OF CHANGE

- An agile process reduces the cost of change because software is released in increments and change can be better controlled within an increment.
- Agility argue that a well-designed agile process flattens the cost of change curve shown in following figure, allowing a software team to accommodate changes late in a software project without dramatic cost and time impact.
- When incremental delivery is coupled with other agile practices such as continuous unit testing and pair programming, the cost of making a change is attenuated.
- Although debate about the degree to which the cost curve flattens is ongoing, there is evidence to suggest that a significant reduction in the cost of change can be achieved.



AGILE PROCESS

- Any agile software process is characterized in a manner that addresses several key assumptions about most software projects:
  1. It is difficult to predict in advance which software requirements will persist and which will change. It is equally difficult to predict how customer priorities will change as the project proceeds.
  2. For many types of software, design and construction are interleaved. That is, both activities should be performed in tandem so that design models are proven as they are created. It is difficult to predict how much design is necessary before construction is used to prove the design.
  3. Analysis, design, construction, and testing are not as predictable.

Principles behind the Agile Manifesto

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face–to–face conversation.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity – the art of maximizing the amount of work not done – is essential.

11. The best architectures, requirements, and designs emerge from self–organizing teams.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

VARIOUS AGILE METHODOLOGIES

Agile model is divided into multiple sub models

- Rational unify process (RUP)
- Adaptive software development (ASD)
- Feature driven development (FDD)
- Crystal clear
- Kanban
- Dynamic software development method
- **Extreme Programming**
- **SCRUM model**

Extreme Programming (XP)

- The most widely used agile process, originally proposed by Kent Beck in 2004. It uses an object-oriented approach.

XP Planning

- Begins with the listening, leads to creation of "user stories" that describes required output, features, and functionality. Customer assigns a value(i.e., a priority) to each story.
- Agile team assesses each story and assigns a cost (development weeks. If more than 3 weeks, customer asked to split into smaller stories)
- Working together, stories are grouped for a deliverable increment next release.
- A commitment (stories to be included, delivery date and other project matters) is made. Three ways:
  1. Either all stories will be implemented in a few weeks,
  2. high priority stories first, or
  3. the riskiest stories will be implemented first.
- After the first increment "project velocity", namely number of stories implemented during the first release is used to help define subsequent delivery dates for other increments. Customers can add stories, delete existing stories, change values of an existing story, split stories as development work proceeds.

XP Design (occurs both before and after coding as refactoring is encouraged)

- Follows the KIS principle (keep it simple) Nothing more nothing less than the story.
- Encourage the use of CRC (class-responsibility-collaborator) cards in an object-oriented context. The only design work product of XP. They identify and organize the classes that are relevant to the current software increment.
- For difficult design problems, suggests the creation of "spike solutions"—a design prototype for that portion is implemented and evaluated.
- Encourages "refactoring"—an iterative refinement of the internal program design. Does not alter the external behavior yet improve the internal structure. Minimize chances of bugs. More efficient, easy to read.

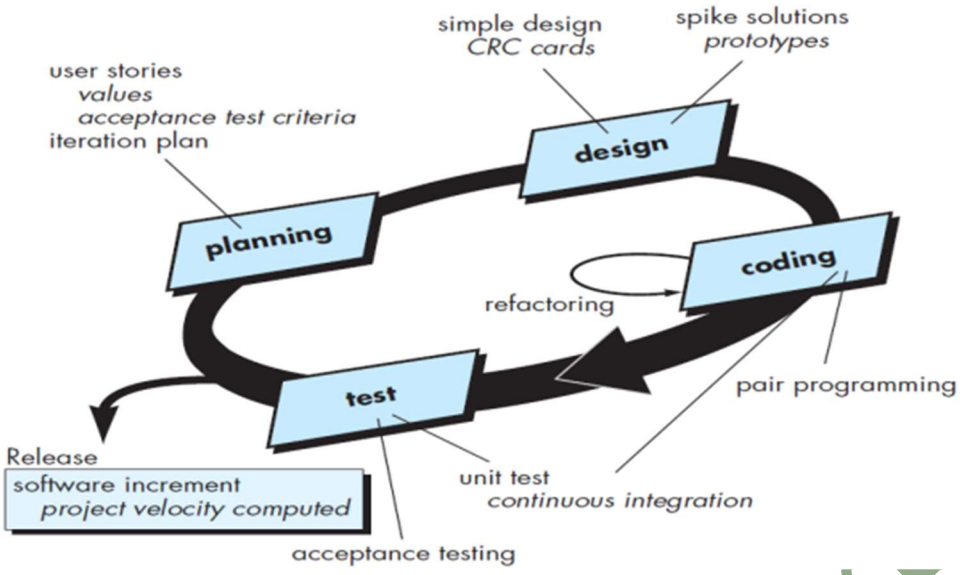
XP Coding

- Recommends the construction of a unit test for a story before coding commences. So implementer can focus on what must be implemented to pass the test.
- Encourages "pair programming". Two people work together at one workstation. Real time problem solving, real time review for quality assurance. Take slightly different roles.

XP Testing

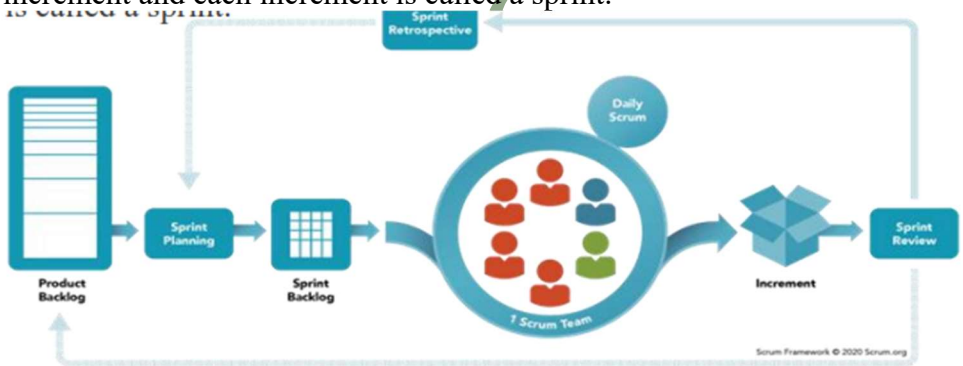
- All unit tests are executed daily and ideally should be automated. Regression tests are conducted to test current and previous components.
- "Acceptance tests" are defined by the customer and executed to assess customer visible functionality

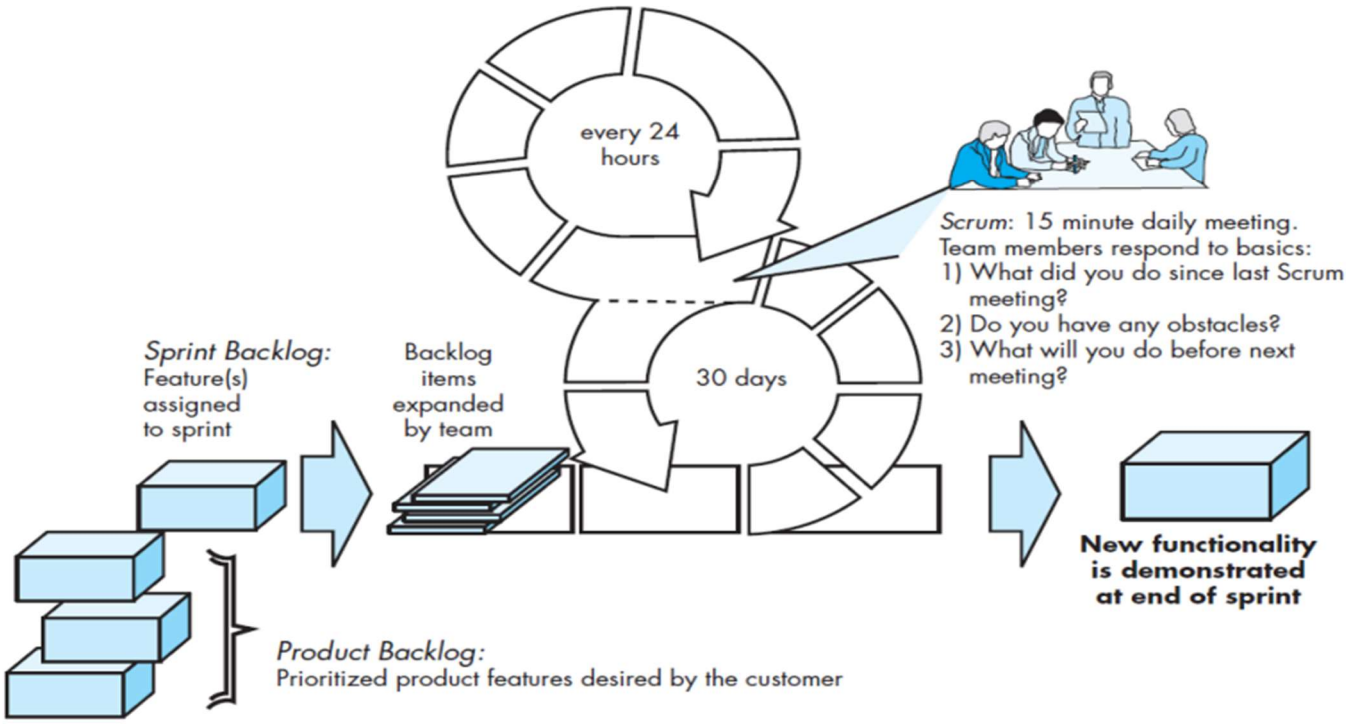
ICSM-C 24B



Scrum Model

- Originally proposed by Schwaber and Beedle SCRUM is agile based model and is derived from rugby game, it is iterative model not a linear sequential model. Total software product will be developed increment by increment and each increment is called a sprint.





The Scrum Events

- **Sprint:** A Sprint is a time-box of one month or less. A new Sprint starts immediately after the completion of the previous Sprint.

- **Sprint Planning:** initiates the Sprint by laying out the work to be performed for the Sprint. This resulting plan is created by the collaborative work of the entire Scrum Team.
- **Daily Scrum:** the purpose of the Daily Scrum is to inspect progress toward the Sprint Goal and adapt the Sprint Backlog as necessary, adjusting the upcoming planned work.
- **Release:** When the product is completed then it goes to the Release stage.
- **Sprint Review:** If the product still has some non-achievable features then it will be checked in this stage and then the product is passed to the Sprint Retrospective stage.

Sprint Retrospective: In this stage quality or status of the product is checked.

Scrum Artifacts

- Scrum's artifacts represent work or value to provide transparency and opportunities for inspection and adaptation. Artifacts defined by Scrum are specifically designed to maximize transparency of key information so that everybody has the same understanding of the artifact. The Scrum Artifacts are:
  - Product Backlog
  - Sprint Backlog
  - Increment
  - Product Backlog

Advantage of using Scrum framework:

- ✓ Scrum framework is fast moving and money efficient.
- ✓ Scrum framework works by dividing the large product into small sub-products. It's like a divide
- ✓ and conquer strategy
- ✓ In Scrum customer satisfaction is very important.
- ✓ Scrum is adaptive in nature because it have short sprint.
- ✓ As Scrum framework rely on constant feedback therefore the quality of product increases in less amount of time

Disadvantage of using Scrum framework:

- ✓ Scrum framework does not allow changes into their sprint.
- ✓ Scrum framework is not fully described model. If you wanna adopt it you need to fill in the
- ✓ framework with your own details like Extreme Programming (XP), Kanban, DSDM.
- ✓ It can be difficult for the Scrum to plan, structure and organize a project that lacks a clear definition. The daily Scrum meetings and frequent reviews require substantial resources.

Approach	Core Idea	Strength	Main Limitation	Typical Use
Agile Methodology	Flexible, iterative development	Adapts to change	Needs customer involvement	Modern software
XP	Engineering-focused Agile method	High code quality	Needs skilled developers	Small–medium projects
Scrum	Agile project management framework	Strong team coordination	Needs discipline	Product development

IICSM-C 24B